# Beyond Accuracy:
# A Deep Dive into AU-PRC and AUC-ROC for Imbalanced Classification

42.515: Machine Learning and Analytics
MTD DS Term 2 / 2026

**Group 2**
Emmanuel 1011048
GunHyun J. 1010966
Ian Thomas Mathew 1010766
Nicole 1011069

Emmanuel 1011048
GunHyun 1010966
Ian Thomas Matthew 1010766
Nicole 1011069
Submission Date: 25th April, 2026
42.515 - Machine Learning and Analytics
MTD DS Term 2/ 2026

---

# Table of Contents

Emmanuel 1011048
GunHyun 1010966
Ian Thomas Matthew 1010766
Nicole 1011069
Submission Date: 25th April, 2026
42.515 - Machine Learning and Analytics
MTD DS Term 2/ 2026

# 1. Motivation

Not every prediction error costs the same. In medical screening, a missed cancer diagnosis can mean a patient goes months without treatment; an unnecessary biopsy is costly and stressful, but recoverable. In fraud detection, allowing a single large transaction to slip through may dwarf the combined annoyance of dozens of blocked legitimate purchases. This asymmetry is baked into nearly every high-stakes classification problem, yet standard evaluation metrics often obscure it entirely.

The situation is further complicated when the class we care about most is also the rarest. Churning customers, fraudulent transactions, and defective components all tend to represent a small fraction of the total data. Under these conditions, a model can achieve deceptively high accuracy while learning almost nothing useful about the minority class. This is a trap that neither accuracy nor AUC-ROC reliably exposes.

## 1.1 Cost of Misclassification

The asymmetry shows up across industries. In clinical settings, a missed diagnosis (false negative) can mean delayed treatment and worse outcomes, whereas an unnecessary follow-up scan (false positive) is inconvenient but not irreversible. For a payment network processing millions of transactions daily, letting even a handful of high-value fraudulent charges through can outweigh the friction of blocking several legitimate purchases for review. In semiconductor manufacturing, a defective chip that reaches a consumer device may trigger recalls costing orders of magnitude more than scrapping a borderline-good component early. The common thread: false negatives compound in ways that false positives rarely do.

## 1.2 The "Accuracy Paradox"

Accuracy fails as a metric the moment classes stop being roughly equal in size. Consider a dataset where 95 out of every 100 patients are healthy: a classifier that predicts healthy for every single person scores 95% accuracy without ever examining a feature. From the accuracy column alone, that model looks excellent. The 5% of sick patients it consistently misses are invisible in the headline number, which is precisely the failure mode that costs lives or revenue.

Emmanuel 1011048
GunHyun 1010966
Ian Thomas Matthew 1010766
Nicole 1011069
Submission Date: 25th April, 2026
42.515 - Machine Learning and Analytics
MTD DS Term 2/ 2026

## 1.3 Decision-Boundary Shift

Minimizing total errors sounds reasonable until you realise what it rewards in an imbalance setting. Logistic regression and most other standard classifiers optimize aggregate loss, so when 95% of the training set belongs to one class, the path of least resistance is to classify nearly everything as that majority class. The decision boundary drifts not because of any bug, but because it is doing exactly what it was asked to do.

Consider the Log Loss function for binary classification:

$$L = -\left[ y \cdot \log(\widehat{p}) + (1-y) \cdot \log(1-\widehat{p}) \right]$$

When most labels y are 0, the $(1-y) \cdot \log(1-\widehat{p})$ term drives nearly all of the gradient. Driving $\widehat{p}$ toward 0 for the whole dataset produces a low aggregate loss while completely ignoring the minority class, and the loss function never notices, because those rare y=1 examples contribute so few terms.

## 1.4 Optimisation & Resource Allocation

Imbalance distorts learning not just at inference time but during training itself. In stochastic gradient descent, each sample contributes an update proportional to its gradient. With a 95:5 split, the majority class generates roughly nineteen updates for every one from the minority. Those minority-class gradients get drowned out, so the model learns majority-class patterns deeply while barely touching the signal that matters. Regularisation methods like Elastic Net compound this: they prune features that do not help overall loss, and features predictive only for the rare class are easily discarded even when they would be diagnostically critical.

Emmanuel 1011048
GunHyun 1010966
Ian Thomas Matthew 1010766
Nicole 1011069
Submission Date: 25th April, 2026
42.515 - Machine Learning and Analytics
MTD DS Term 2/ 2026

# 2. Context: Related Topics

Understanding AU-PRC and AUC-ROC does not exist in isolation. The table below positions this topic within the broader machine learning landscape.

| Category | Topic | Explanation |
|---|---|---|
| **Precedent** | Confusion Matrix | You must understand TP, FP, TN, and FN before calculating these curves. |
| **Successor** | Cost-Sensitive Learning | Once you know the model is failing on the minority class, you apply penalties to fix it during training. |
| **Tangential** | Calibration Curves | While AUC measures ranking ability, calibration measures whether a predicted 80% probability actually occurs 80% of the time. |

## 2.1 Precedent - Confusion Matrix

Both the ROC and Precision-Recall curves are derived from the same underlying four-cell table, the confusion matrix. Every point on either curve corresponds to a specific classification threshold, and moving that threshold sweeps through different TP/FP/TN/FN combinations. The reason Recall ($TP \div actual\ positives$) appears as an axis in both curves is not coincidental. It is the one metric that directly answers the question a practitioner cares most about in an imbalance problem. The question is, of all the real positives, how many did we catch?

## 2.2 Successor - Cost-Sensitive Learning

Once AU-PRC reveals that the model is underperforming on the minority class, the natural next question is, what do we do about it? Cost-sensitive learning reweights the loss so that a missed positive is penalised more heavily than a false alarm. It effectively tells that optimiser that these errors are not equivalent. Data-level approaches like SMOTE take a different route, synthetically oversampling the minority class so that the gradient is exposed to more minority-class signals during training. Finally, threshold tuning acknowledges that deploying a classifier requires picking a single operating point. Where the AU-PRC curve tells you how good your options are, and threshold optimization tells you which options to actually use.

Emmanuel 1011048
GunHyun 1010966
Ian Thomas Matthew 1010766
Nicole 1011069
Submission Date: 25th April, 2026
42.515 - Machine Learning and Analytics
MTD DS Term 2/ 2026

---

### 2.3 Tangential - Calibration Curves

AUC measures ranking quality. The question is, does the model consistently assign higher scores to real positives than to negatives? Calibration asks a subtler question, when the model outputs a probability of 0.80, does that number actually mean the event occurs 80% of the time in practice? A model can rank perfectly (high AUC) while being wildly miscalibrated, which is common with tree ensembles and neural networks. In clinical decision support, where a physician may act differently on a 30% versus 60% risk estimate, this gap matters enormously. Calibration curves (reliability diagrams) complement AU-PRC by checking whether probability outputs can be trusted at face value, not just used for ranking.

## 3. Theory

### 3.1 AU - PRC (Area Under the Precision-Recall Curve)

The Precision-Recall Curve plots two quantities against each other as the classification threshold sweeps from 0 to 1. Precision answers, of everything we flagged positive, what fraction actually was? Recall answers, of everything that actually was positive, what fraction did we flag? The tension between these two is most visible in imbalance settings. For example, a loose threshold catches most true positives (high recall) but drowns them in false alarms (low precision).

The AU-PRC is computed as Average Precision (AP):

$$AP = \sum_{n=1}^{N} \left( R_n - R_{n-1} \right) \times P_n$$

Where $P_n$ and $R_n$ are the precision and recall at the n-th threshold. This is exactly what the sklearn.metrics.average_precision_score computes.

Emmanuel 1011048
GunHyun 1010966
Ian Thomas Matthew 1010766
Nicole 1011069
Submission Date: 25th April, 2026
42.515 - Machine Learning and Analytics
MTD DS Term 2/ 2026

---

One detail that trips people up is that the random baseline for AU-PRC is not 0.5. It equals the class prevalence. At a 5% minority rate, a random classifier scores roughly 0.05, meaning a model reporting AU-PRC of 0.35 on such data is actually doing substantial work, even though 0.35 sounds unimpressive in isolation.

## 3.2 AUC - ROC (Area Under the Receiver Operating Characteristic Curve)

The ROC curve plots the **True Positive Rate (TPR)** against the **False Positive Rate (FPR)** at various threshold settings.

- **TPR (Recall/Sensitivity):** $\frac{TP}{TP + FN}$
- **FPR (1 - Specificity):** $\frac{FP}{FP + TN}$

Hanley & McNeil (1982) showed that AUC-ROC has a clean probabilistic meaning, where it is the probability that a randomly drawn positive example receives a higher model score than a randomly drawn negative example. This interpretation is appealing because it does not depend on any threshold choice. The random baseline is always 0.5 regardless of how skewed the class distribution is, which, as we will see, is also the source of its misleading behaviour on imbalanced data.

## 3.3 The Precision-Recall Trade-off

Raising recall almost always comes at the cost of precision, and vice versa. Loosening the threshold to catch every last true positive inevitably sweeps in more false positives; tightening it to keep precision high means letting some real positives slip through. There is no free lunch here, the curve itself is a visualisation of every possible trade-off, and the AU-PRC summarises how good those trade-offs are across the board.

Emmanuel 1011048
GunHyun 1010966
Ian Thomas Matthew 1010766
Nicole 1011069
Submission Date: 25th April, 2026
42.515 - Machine Learning and Analytics
MTD DS Term 2/ 2026

A practical analogy, think of a hospital triage nurse deciding which patients need immediate attention. Flagging everyone as critical (maximum recall) ensures no serious case is missed out but overwhelms the ICU with unnecessary admissions. Flagging only the most obviously severe cases (maximum precision) keeps resources focused but risks missing patients who deteriorate quickly. The right operating point depends on bed availability, staff capacity, and the relative cost of each error type. This is exactly the kind of domain knowledge the AU-PRC curve is designed to inform.

This relationship is often summarised by the **F1-Score**, which is the harmonic mean of Precision and Recall:

$$F_1 = 2 \cdot \frac{P \cdot R}{P + R}$$

The harmonic mean is used deliberately: it penalises extreme imbalance between the two values far more than an arithmetic average would. A model with Precision = 1.0 and Recall = 0.0 would receive an arithmetic mean of 0.5, but an F1 of 0 which correctly reflects that the model is not useful.

## 3.4 Why AUC-ROC Inflates Under Class Imbalance

The inflation comes from a single denominator difference. FPR is computed as $FP \div (FP + TN)$. When the majority class is large, TN is enormous. So even a model generating thousands of false positives keeps FPR low, because those false positives are divided by an even larger pool of true negatives. The ROC curve barely flinches. Precision ( $TP \div (TP + FP)$) has no such cushion. The moment FP grows, precision drops, full stop. This is why two classifiers can look almost identical on ROC space yet differ dramatically on Precision-Recall plot.

Davis & Goadrich (2006) formalised this relationship, where dominance in ROC space implies dominance in PR space, but not the other way around. This means PR space offers strictly finer discrimination when classes are skewed. Saito & Rehmsmeier (2015) extended this with empirical evidence across multiple datasets, showing that AUC-ROC can stay above 0.9 on problems where AU-PRC tells a sobering different story.

Emmanuel 1011048
GunHyun 1010966
Ian Thomas Matthew 1010766
Nicole 1011069
Submission Date: 25th April, 2026
42.515 - Machine Learning and Analytics
MTD DS Term 2/ 2026

| | AUC-ROC | AU-PRC |
|---|---|---|
| Uses True Negatives? | Yes - in FPR denominator | No - TN never appears |
| Random baseline | 0.5 always | Equals class prevalence (e.g. 0.05 at 5% imbalance) |
| Behavior under imbalance | Inflated - appears too optimistic | Accurate - reflects true minority-class performance |
| Best used when | Classes are roughly balanced | Minority class performance is the primary concern |

# 4. Application

The accompanying notebook (au_prc_vs_auc_roc_churn_models.ipynb) works through the divergence between these two metrics on a customer churn dataset, where churners represent the minority class. We train three models, Logistic Regression, LightGBM, and XGBoost to evaluate side by side. The churn label is ChurnStatus (Yes/No), with the positive class being churners.

Preprocessing follows a straightforward pipeline, where median imputation and standard scaling for numerics, mode imputation and one-hot-encoding for categoricals, with a stratified train-validation split to preserve class ratios. We then compute roc_auc_score and average_precision_score from scikit-learn for each model and plot both curves side by side. To sharpen the contrast, we also run the same models on a resampled version of the data with a 10% churn rate, where the gap between the two metrics becomes visibly wider.

Across all three models, AUC-ROC stays relatively stable as imbalance increases, while AU-PRC falls noticeable, where in some runs by more than 0.15 points. This mirrors exactly the structural argument in Section 3.4 and illustrates why reporting only AUC-ROC on a churn or fraud problem can give an overly optimistic picture of real performance.

Emmanuel 1011048
GunHyun 1010966
Ian Thomas Matthew 1010766
Nicole 1011069
Submission Date: 25th April, 2026
42.515 - Machine Learning and Analytics
MTD DS Term 2/ 2026

# 5. References

1. Davis, J., & Goadrich, M. (2006). The relationship between Precision-Recall and ROC curves. *Proceedings of the 23rd International Conference on Machine Learning (ICML 2006)*, pp. 233–240.
2. Saito, T., & Rehmsmeier, M. (2015). The Precision-Recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE*, 10(3), e0118432.
3. Hanley, J. A., & McNeil, B. J. (1982). The meaning and use of the area under a receiver operating characteristic (ROC) curve. *Radiology*, 143(1), 29–36.
4. He, H., & Garcia, E. A. (2009). Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9), 1263–1284.
5. Lobo, J. M., *et al.* (2024). A closer look at AUROC and AUPRC under class imbalance. *arXiv preprint*, arXiv:2401.06091.
6. Chicco, D., & Jurman, G. (2020). The advantages of the Matthews Correlation Coefficient (MCC) over F1 score and accuracy in binary classification evaluation. *BMC Genomics*, 21, 6.

Emmanuel 1011048
GunHyun 1010966
Ian Thomas Matthew 1010766
Nicole 1011069
Submission Date: 25th April, 2026
42.515 - Machine Learning and Analytics
MTD DS Term 2/ 2026

---

# 6. Appendix

## AU-PRC vs AUC-ROC on a Customer Churn Dataset

This notebook compares **AU-PRC** and **AUC-ROC** on a real tabular dataset using:

- **Logistic Regression**
- **LightGBM**
- **XGBoost**

It uses the provided files:

- `train.csv`
- `test.csv`

## Goals

1. Use the provided churn dataset instead of synthetic data.
2. Build an end-to-end preprocessing + modeling workflow in `scikit-learn`.
3. Compare **ROC-AUC** and **Average Precision (AU-PRC style summary)**.
4. Visualize ROC and Precision-Recall curves.
5. Show why PR-based evaluation can become more informative on imbalanced data.
6. Optionally create a **more imbalanced version** of the same dataset for a stronger demonstration.

> Note: In scikit-learn, `average_precision_score` is the standard summary metric for Precision-Recall performance.
> It is commonly used in assignments as a practical AU-PRC-style summary.

Emmanuel 1011048
GunHyun 1010966
Ian Thomas Matthew 1010766
Nicole 1011069
Submission Date: 25th April, 2026
42.515 - Machine Learning and Analytics
MTD DS Term 2/ 2026

---

```
import warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from pathlib import Path

from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    roc_curve,
    roc_auc_score,
    precision_recall_curve,
    average_precision_score,
    confusion_matrix,
    classification_report,
    ConfusionMatrixDisplay
)

RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)
```

## 1. Load the provided dataset

This notebook assumes `train.csv` and `test.csv` are in the same folder as the notebook.

- `train.csv` contains the target column: **ChurnStatus**
- `test.csv` is included for optional final inference

Emmanuel 1011048
GunHyun 1010966
Ian Thomas Matthew 1010766
Nicole 1011069
Submission Date: 25th April, 2026
42.515 - Machine Learning and Analytics
MTD DS Term 2/ 2026

---

```
train_path = Path("train.csv")
test_path = Path("test.csv")

if not train_path.exists() or not test_path.exists():
    raise FileNotFoundError(
        "Make sure both 'train.csv' and 'test.csv' are in the same folder as this notebook."
    )

train_df = pd.read_csv(train_path)
test_df = pd.read_csv(test_path)

print("Train shape:", train_df.shape)
print("Test shape: ", test_df.shape)

display(train_df.head())
```

```
Train shape: (70430, 45)
Test shape:  (14086, 45)
```

| | CustomerID | UserGender | UserAge | YoungAdultFlag | RetireeStatus | Married | Dependents | NumberofDependents | Country |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 1610a102a7854c5d | Male | 71 | No | Yes | Yes | Yes | 1 | United States |
| 1 | 7b4b47a335af4741 | Male | 77 | No | Yes | Yes | No | 0 | United States |
| 2 | 81465cd8c020404b | Male | 78 | No | Yes | Yes | No | 0 | United States |
| 3 | 8fb085ca69574c4c | Male | 74 | No | Yes | No | No | 0 | United States |
| 4 | d2ca70e00e1d419f | Male | 71 | No | Yes | Yes | Yes | 1 | United States |

5 rows × 45 columns

## 2. Quick dataset audit

We inspect:

- target balance
- column types
- missing values

The target for this notebook is **ChurnStatus**:

- `Yes` → 1
- `No` → 0

Emmanuel 1011048
GunHyun 1010966
Ian Thomas Matthew 1010766
Nicole 1011069
Submission Date: 25th April, 2026
42.515 - Machine Learning and Analytics
MTD DS Term 2/ 2026

---

```
target_col = "ChurnStatus"
id_col = "CustomerID"

if target_col not in train_df.columns:
    raise ValueError(f"Expected target column '{target_col}' not found in train.csv")

df = train_df.copy()
df[target_col] = df[target_col].map({"No": 0, "Yes": 1})

print("Target distribution:")
display(df[target_col].value_counts().rename(index={0: "No churn", 1: "Churn"}))

print("Target proportions:")
display(df[target_col].value_counts(normalize=True).rename(index={0: "No churn", 1: "Churn"}))

missing_df = pd.DataFrame({
    "missing_count": df.isna().sum(),
    "missing_share": df.isna().mean()
}).sort_values("missing_share", ascending=False)

display(missing_df.head(15))
```

Emmanuel 1011048
GunHyun 1010966
Ian Thomas Matthew 1010766
Nicole 1011069
Submission Date: 25th April, 2026
42.515 - Machine Learning and Analytics
MTD DS Term 2/ 2026

---

```
Target distribution:
```

| ChurnStatus | count |
|---|---|
| No churn | 50155 |
| Churn | 20275 |

**dtype:** int64

```
Target proportions:
```

| ChurnStatus | proportion |
|---|---|
| No churn | 0.712126 |
| Churn | 0.287874 |

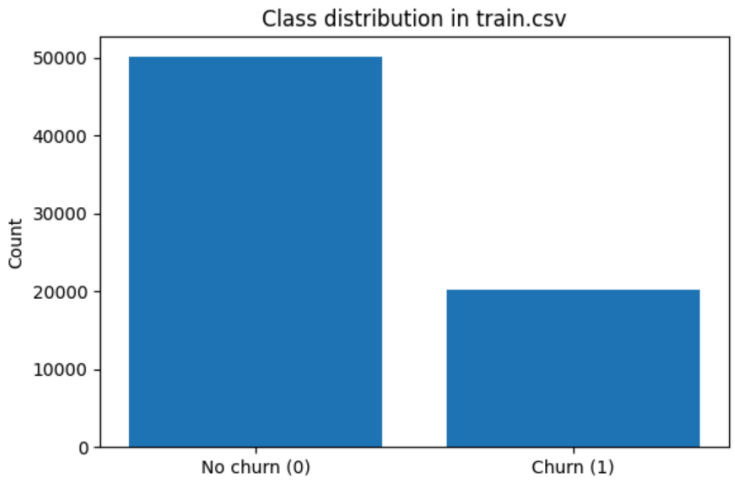
**dtype:** float64

| | missing_count | missing_share |
|---|---|---|
| Offer | 48221 | 0.684666 |
| InternetType | 17018 | 0.241630 |
| PrioritySupport | 12383 | 0.175820 |
| DigitalInvoicing | 12173 | 0.172838 |
| CustomerID | 0 | 0.000000 |
| Married | 0 | 0.000000 |
| Dependents | 0 | 0.000000 |
| NumberofDependents | 0 | 0.000000 |
| Country | 0 | 0.000000 |
| UserGender | 0 | 0.000000 |
| UserAge | 0 | 0.000000 |
| YoungAdultFlag | 0 | 0.000000 |
| RetireeStatus | 0 | 0.000000 |
| Latitude | 0 | 0.000000 |
| AreaCode | 0 | 0.000000 |

Emmanuel 1011048
GunHyun 1010966
Ian Thomas Matthew 1010766
Nicole 1011069
Submission Date: 25th April, 2026
42.515 - Machine Learning and Analytics
MTD DS Term 2/ 2026

---

```python
# Class distribution plot
target_counts = df[target_col].value_counts().sort_index()

plt.figure(figsize=(6, 4))
plt.bar(["No churn (0)", "Churn (1)"], target_counts.values)
plt.title("Class distribution in train.csv")
plt.ylabel("Count")
plt.tight_layout()
plt.show()
```




```python
print(target_counts)
```

```
ChurnStatus
0    50155
1    20275
Name: count, dtype: int64
```

## 3. Feature setup

We drop:

- `CustomerID` as an identifier
- `ChurnStatus` from the features

Then we build preprocessing pipelines:

- numeric: median imputation + scaling
- categorical: most-frequent imputation + one-hot encoding

Emmanuel 1011048
GunHyun 1010966
Ian Thomas Matthew 1010766
Nicole 1011069
Submission Date: 25th April, 2026
42.515 - Machine Learning and Analytics
MTD DS Term 2/ 2026

```
feature_df = df.drop(columns=[target_col])
if id_col in feature_df.columns:
    feature_df = feature_df.drop(columns=[id_col])

X = feature_df.copy()
y = df[target_col].copy()

numeric_features = X.select_dtypes(include=["int64", "float64"]).columns.tolist()
categorical_features = X.select_dtypes(include=["object"]).columns.tolist()

print("Number of numeric features:", len(numeric_features))
print("Number of categorical features:", len(categorical_features))

numeric_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer([
    ("num", numeric_transformer, numeric_features),
    ("cat", categorical_transformer, categorical_features)
])
```

```
Number of numeric features: 17
Number of categorical features: 26
```

## 4. Train-validation split

We use a stratified split so the class distribution is preserved in both sets.

```
X_train, X_valid, y_train, y_valid = train_test_split(
    X, y,
    test_size=0.25,
    stratify=y,
    random_state=RANDOM_STATE
)

print("Train shape:", X_train.shape)
print("Validation shape:", X_valid.shape)
print("Train positive rate:", round(y_train.mean(), 4))
print("Validation positive rate:", round(y_valid.mean(), 4))
```

```
Train shape: (52822, 43)
Validation shape: (17608, 43)
Train positive rate: 0.2879
Validation positive rate: 0.2879
```

Emmanuel 1011048
GunHyun 1010966
Ian Thomas Matthew 1010766
Nicole 1011069
Submission Date: 25th April, 2026
42.515 - Machine Learning and Analytics
MTD DS Term 2/ 2026

---

```
# !pip install lightgbm xgboost
available_models = {}

# Logistic Regression is always included
available_models["Logistic Regression"] = LogisticRegression(
    max_iter=2000,
    random_state=RANDOM_STATE
)

# LightGBM
try:
    from lightgbm import LGBMClassifier
    available_models["LightGBM"] = LGBMClassifier(
        n_estimators=300,
        learning_rate=0.05,
        num_leaves=31,
        random_state=RANDOM_STATE,
        verbose=-1
    )
    print("LightGBM loaded successfully.")
except Exception as e:
    print("LightGBM not available:", e)

# XGBoost
try:
    from xgboost import XGBClassifier
    available_models["XGBoost"] = XGBClassifier(
        n_estimators=300,
        learning_rate=0.05,
        max_depth=6,
        subsample=0.9,
        colsample_bytree=0.9,
        eval_metric="logloss",
        random_state=RANDOM_STATE
    )
    print("XGBoost loaded successfully.")
except Exception as e:
    print("XGBoost not available:", e)

print("\nModels to evaluate:", list(available_models.keys()))
```

```
LightGBM loaded successfully.
XGBoost loaded successfully.

Models to evaluate: ['Logistic Regression', 'LightGBM', 'XGBoost']
```

## 6. Helper functions for evaluation

We evaluate each model using:

- ROC-AUC
- Average Precision
- confusion matrix
- classification report
- ROC curve
- Precision-Recall curve

Emmanuel 1011048
GunHyun 1010966
Ian Thomas Matthew 1010766
Nicole 1011069
Submission Date: 25th April, 2026
42.515 - Machine Learning and Analytics
MTD DS Term 2/ 2026

```python
def build_pipeline(model):
    return Pipeline([
        ("preprocessor", preprocessor),
        ("model", model)
    ])


def evaluate_model(model_name, model, X_train, X_valid, y_train, y_valid):
    pipe = build_pipeline(model)
    pipe.fit(X_train, y_train)

    y_score = pipe.predict_proba(X_valid)[:, 1]
    y_pred = pipe.predict(X_valid)

    fpr, tpr, _ = roc_curve(y_valid, y_score)
    precision, recall, _ = precision_recall_curve(y_valid, y_score)

    roc_auc = roc_auc_score(y_valid, y_score)
    avg_precision = average_precision_score(y_valid, y_score)

    metrics_row = {
        "model": model_name,
        "roc_auc": roc_auc,
        "avg_precision": avg_precision,
        "accuracy": (y_pred == y_valid).mean(),
        "positive_rate_valid": y_valid.mean()
    }

    return {
        "model_name": model_name,
        "pipeline": pipe,
        "y_pred": y_pred,
        "y_score": y_score,
        "fpr": fpr,
        "tpr": tpr,
        "precision": precision,
        "recall": recall,
        "roc_auc": roc_auc,
        "avg_precision": avg_precision,
        "confusion_matrix": confusion_matrix(y_valid, y_pred),
        "report_df": pd.DataFrame(classification_report(y_valid, y_pred, output_dict=True)).transpose(),
        "metrics_row": metrics_row
    }
```

Emmanuel 1011048
GunHyun 1010966
Ian Thomas Matthew 1010766
Nicole 1011069
Submission Date: 25th April, 2026
42.515 - Machine Learning and Analytics
MTD DS Term 2/ 2026

```
def plot_curve_set(results, title_suffix="Validation set"):
    plt.figure(figsize=(7, 5))
    for res in results:
        plt.plot(res["fpr"], res["tpr"], label=f'{res["model_name"]} (AUC={res["roc_auc"]:.3f})')
    plt.plot([0, 1], [0, 1], linestyle="--", label="Random baseline")
    plt.xlabel("False Positive Rate")
    plt.ylabel("True Positive Rate")
    plt.title(f"ROC Curves - {title_suffix}")
    plt.legend()
    plt.tight_layout()
    plt.show()

    baseline_precision = results[0]["metrics_row"]["positive_rate_valid"]

    plt.figure(figsize=(7, 5))
    for res in results:
        plt.plot(res["recall"], res["precision"], label=f'{res["model_name"]} (AP={res["avg_precision"]:.3f})')
    plt.axhline(y=baseline_precision, linestyle="--", label=f"Baseline precision = {baseline_precision:.3f}")
    plt.xlabel("Recall")
    plt.ylabel("Precision")
    plt.title(f"Precision-Recall Curves - {title_suffix}")
    plt.legend()
    plt.tight_layout()
    plt.show()
```

## 7. Train and compare models on the original dataset

```
results = []

for model_name, model in available_models.items():
    print(f"Training {model_name}...")
    res = evaluate_model(model_name, model, X_train, X_valid, y_train, y_valid)
    results.append(res)

metrics_df = pd.DataFrame([res["metrics_row"] for res in results]).sort_values(
    by=["avg_precision", "roc_auc"], ascending=False
)

display(metrics_df)
```

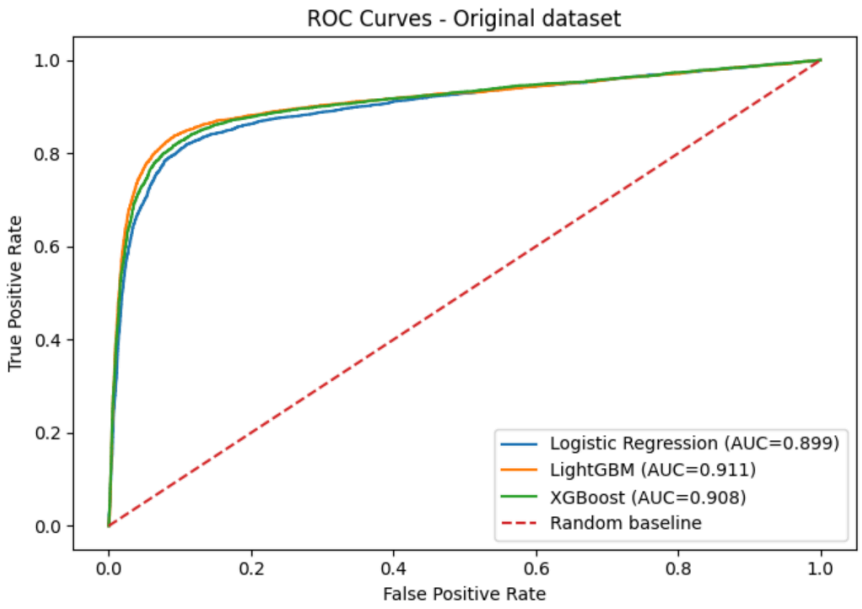
```
Training Logistic Regression...
Training LightGBM...
Training XGBoost...
```

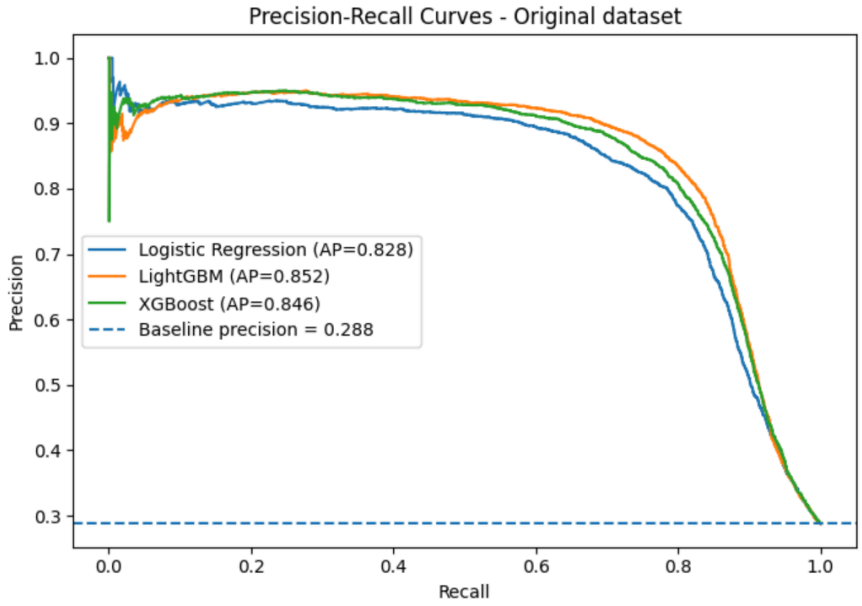
| | model | roc_auc | avg_precision | accuracy | positive_rate_valid |
|---|---|---|---|---|---|
| **1** | LightGBM | 0.910634 | 0.852094 | 0.898058 | 0.287881 |
| **2** | XGBoost | 0.907914 | 0.846129 | 0.890902 | 0.287881 |
| **0** | Logistic Regression | 0.899210 | 0.828398 | 0.881758 | 0.287881 |

Emmanuel 1011048
GunHyun 1010966
Ian Thomas Matthew 1010766
Nicole 1011069
Submission Date: 25th April, 2026
42.515 - Machine Learning and Analytics
MTD DS Term 2/ 2026

---

```
plot_curve_set(results, title_suffix="Original dataset")
```



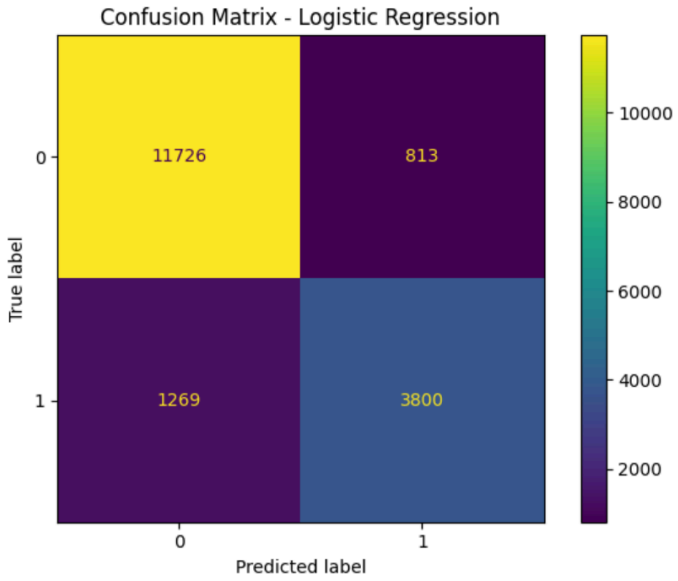
Emmanuel 1011048
GunHyun 1010966
Ian Thomas Matthew 1010766
Nicole 1011069
Submission Date: 25th April, 2026
42.515 - Machine Learning and Analytics
MTD DS Term 2/ 2026

---

## 8. Confusion matrices and reports

```
for res in results:
    print("=" * 90)
    print(res["model_name"])
    ConfusionMatrixDisplay(res["confusion_matrix"]).plot()
    plt.title(f'Confusion Matrix - {res["model_name"]}')
    plt.tight_layout()
    plt.show()
    display(res["report_df"])
```

```
==========================================================================================
Logistic Regression
```

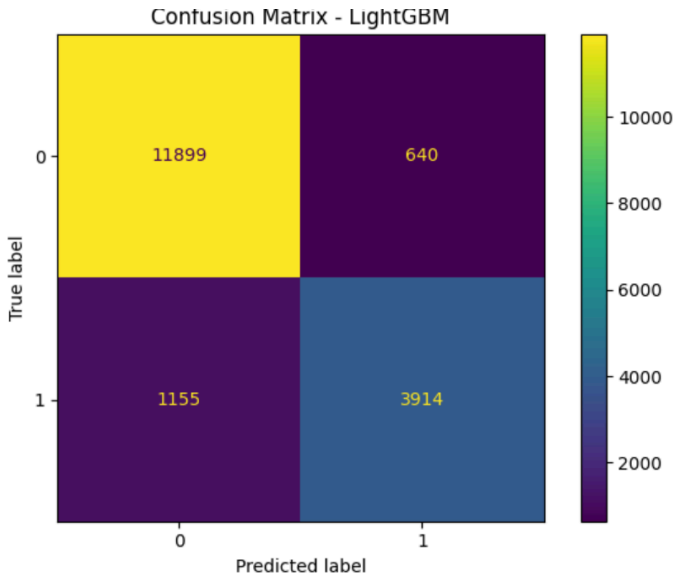



| | precision | recall | f1-score | support |
|---|---|---|---|---|
| **0** | 0.902347 | 0.935162 | 0.918462 | 12539.000000 |
| **1** | 0.823759 | 0.749655 | 0.784962 | 5069.000000 |
| **accuracy** | 0.881758 | 0.881758 | 0.881758 | 0.881758 |
| **macro avg** | 0.863053 | 0.842409 | 0.851712 | 17608.000000 |
| **weighted avg** | 0.879723 | 0.881758 | 0.880030 | 17608.000000 |

```
==========================================================================================
LightGBM
```

Emmanuel 1011048
GunHyun 1010966
Ian Thomas Matthew 1010766
Nicole 1011069
Submission Date: 25th April, 2026
42.515 - Machine Learning and Analytics
MTD DS Term 2/ 2026

---




| | precision | recall | f1-score | support |
|---|---|---|---|---|
| **0** | 0.911521 | 0.948959 | 0.929864 | 12539.000000 |
| **1** | 0.859464 | 0.772144 | 0.813468 | 5069.000000 |
| **accuracy** | 0.898058 | 0.898058 | 0.898058 | 0.898058 |
| **macro avg** | 0.885493 | 0.860552 | 0.871666 | 17608.000000 |
| **weighted avg** | 0.896535 | 0.898058 | 0.896356 | 17608.000000 |

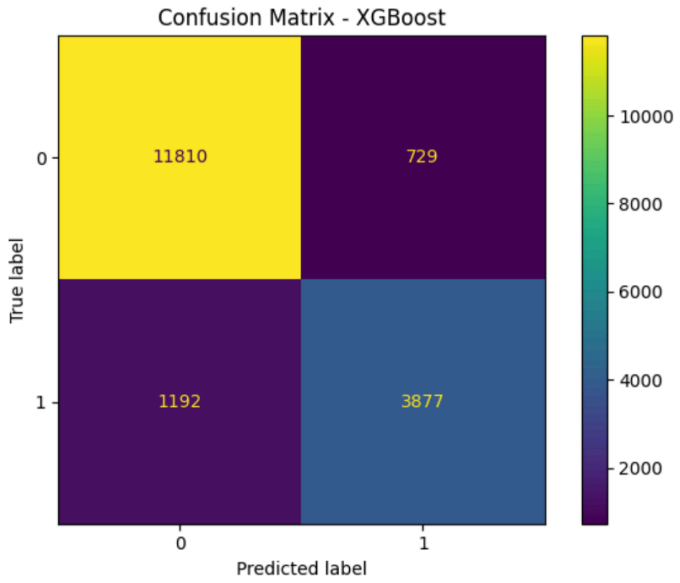
==========

XGBoost

Emmanuel 1011048
GunHyun 1010966
Ian Thomas Matthew 1010766
Nicole 1011069
Submission Date: 25th April, 2026
42.515 - Machine Learning and Analytics
MTD DS Term 2/ 2026

---

...




| | precision | recall | f1-score | support |
|---|---|---|---|---|
| **0** | 0.908322 | 0.941861 | 0.924788 | 12539.000000 |
| **1** | 0.841728 | 0.764845 | 0.801447 | 5069.000000 |
| **accuracy** | 0.890902 | 0.890902 | 0.890902 | 0.890902 |
| **macro avg** | 0.875025 | 0.853353 | 0.863117 | 17608.000000 |
| **weighted avg** | 0.889151 | 0.890902 | 0.889280 | 17608.000000 |

Emmanuel 1011048
GunHyun 1010966
Ian Thomas Matthew 1010766
Nicole 1011069
Submission Date: 25th April, 2026
42.515 - Machine Learning and Analytics
MTD DS Term 2/ 2026

---

## 9. Why AU-PRC may still be important here

This dataset is imbalanced, but not extremely so.
To make the difference between ROC-AUC and AU-PRC more visible, we create a **more imbalanced version of the same dataset** by reducing the positive class.

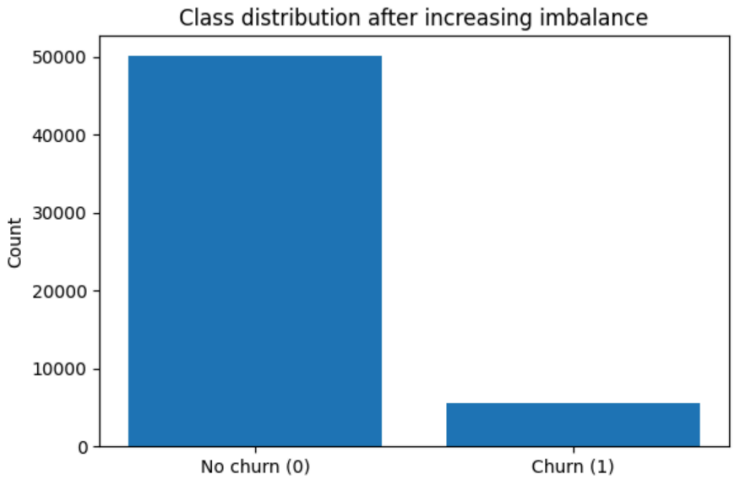
This keeps the assignment grounded in the provided data while making the metric comparison clearer.

```
positive_df = df[df[target_col] == 1].copy()
negative_df = df[df[target_col] == 0].copy()

desired_positive_rate = 0.10
n_neg = len(negative_df)
n_pos_needed = int((desired_positive_rate * n_neg) / (1 - desired_positive_rate))

positive_downsampled = positive_df.sample(
    n=min(n_pos_needed, len(positive_df)),
    random_state=RANDOM_STATE
)

imbalanced_df = pd.concat([negative_df, positive_downsampled], axis=0).sample(
    frac=1.0,
    random_state=RANDOM_STATE
).reset_index(drop=True)

print("Original positive rate:", round(df[target_col].mean(), 4))
print("New positive rate:     ", round(imbalanced_df[target_col].mean(), 4))

plt.figure(figsize=(6, 4))
counts = imbalanced_df[target_col].value_counts().sort_index()
plt.bar(["No churn (0)", "Churn (1)"], counts.values)
plt.title("Class distribution after increasing imbalance")
plt.ylabel("Count")
plt.tight_layout()
plt.show()
```

Emmanuel 1011048
GunHyun 1010966
Ian Thomas Matthew 1010766
Nicole 1011069
Submission Date: 25th April, 2026
42.515 - Machine Learning and Analytics
MTD DS Term 2/ 2026

---

```
Original positive rate: 0.2879
New positive rate:      0.1
```



Emmanuel 1011048
GunHyun 1010966
Ian Thomas Matthew 1010766
Nicole 1011069
Submission Date: 25th April, 2026
42.515 - Machine Learning and Analytics
MTD DS Term 2/ 2026

```
X_imb = imbalanced_df.drop(columns=[target_col])
if id_col in X_imb.columns:
    X_imb = X_imb.drop(columns=[id_col])
y_imb = imbalanced_df[target_col]

X_train_imb, X_valid_imb, y_train_imb, y_valid_imb = train_test_split(
    X_imb, y_imb,
    test_size=0.25,
    stratify=y_imb,
    random_state=RANDOM_STATE
)

print("Train positive rate:", round(y_train_imb.mean(), 4))
print("Validation positive rate:", round(y_valid_imb.mean(), 4))
```

```
Train positive rate: 0.1
Validation positive rate: 0.1
```

## 10. Re-run the comparison on the more imbalanced version

```
results_imb = []

for model_name, model in available_models.items():
    print(f"Training {model_name} on more imbalanced data...")
    res = evaluate_model(model_name, model, X_train_imb, X_valid_imb, y_train_imb, y_valid_imb)
    results_imb.append(res)

metrics_df_imb = pd.DataFrame([res["metrics_row"] for res in results_imb]).sort_values(
    by=["avg_precision", "roc_auc"], ascending=False
)

display(metrics_df_imb)
```

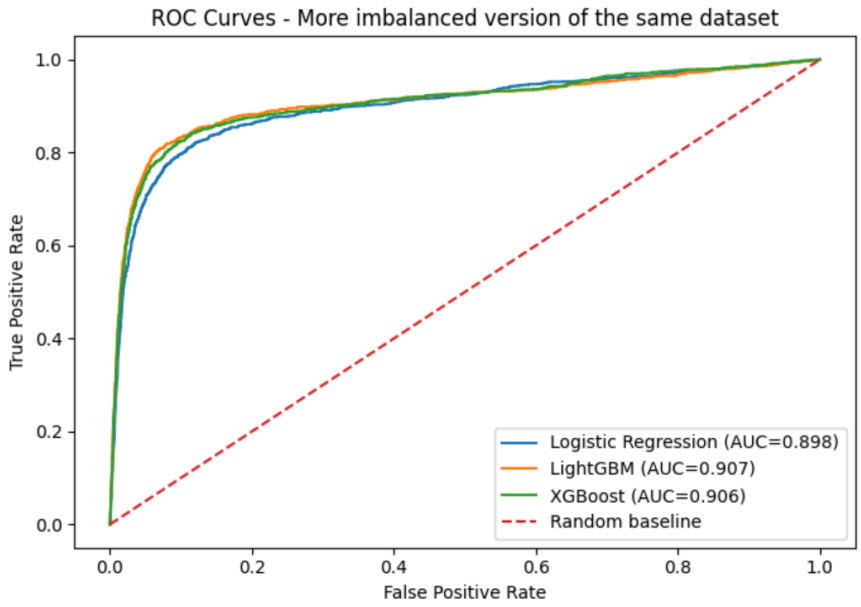
```
Training Logistic Regression on more imbalanced data...
Training LightGBM on more imbalanced data...
Training XGBoost on more imbalanced data...
```

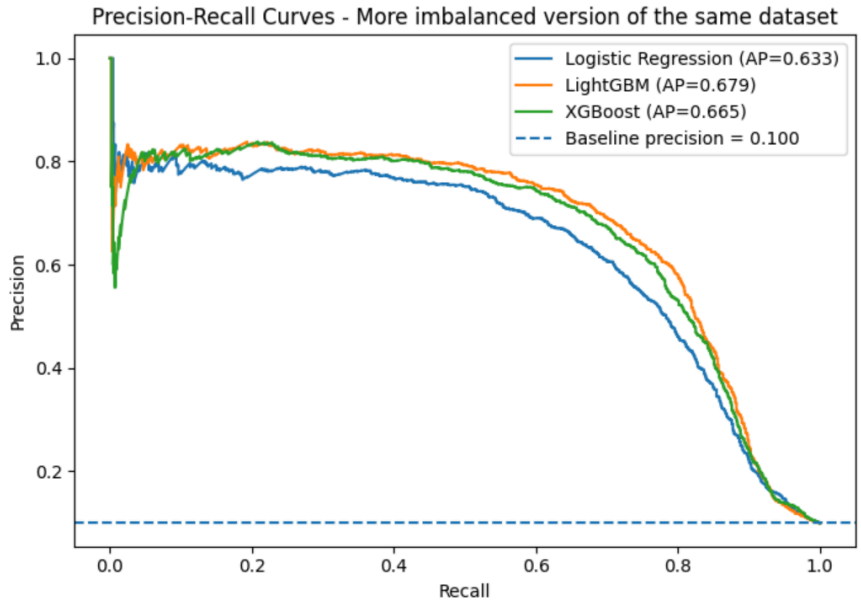
| | model | roc_auc | avg_precision | accuracy | positive_rate_valid |
|---|---|---|---|---|---|
| 1 | LightGBM | 0.906765 | 0.678692 | 0.939779 | 0.099986 |
| 2 | XGBoost | 0.906017 | 0.665281 | 0.936836 | 0.099986 |
| 0 | Logistic Regression | 0.898170 | 0.632826 | 0.931453 | 0.099986 |

Emmanuel 1011048
GunHyun 1010966
Ian Thomas Matthew 1010766
Nicole 1011069
Submission Date: 25th April, 2026
42.515 - Machine Learning and Analytics
MTD DS Term 2/ 2026

---

```
plot_curve_set(results_imb, title_suffix="More imbalanced version of the same dataset")
```



Emmanuel 1011048
GunHyun 1010966
Ian Thomas Matthew 1010766
Nicole 1011069
Submission Date: 25th April, 2026
42.515 - Machine Learning and Analytics
MTD DS Term 2/ 2026

---

## 11. Side-by-side metric comparison

This table makes the assignment argument clearer:

- ROC-AUC may remain fairly high
- Average Precision often changes more noticeably when the positive class becomes rarer

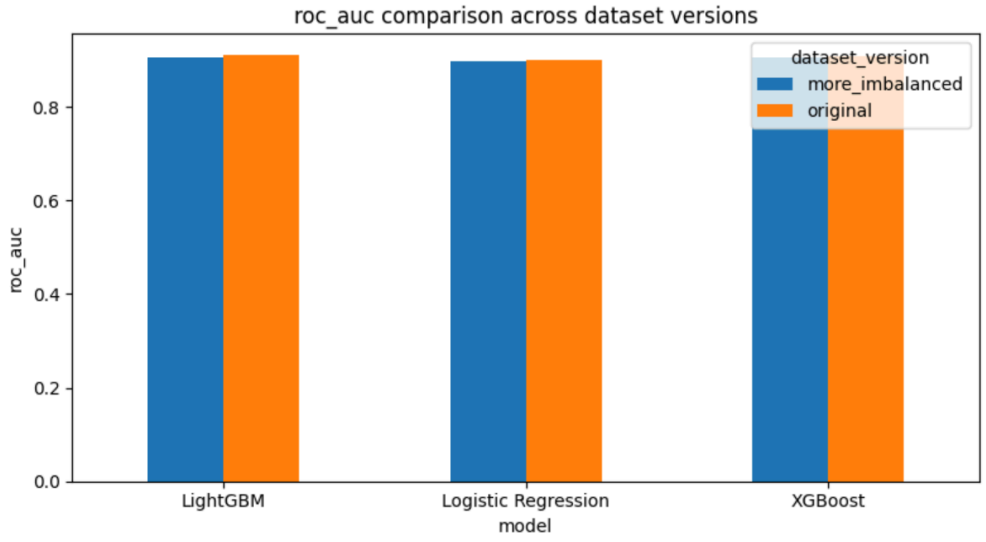
```
original_metrics = metrics_df[["model", "roc_auc", "avg_precision"]].copy()
original_metrics["dataset_version"] = "original"

imbalanced_metrics = metrics_df_imb[["model", "roc_auc", "avg_precision"]].copy()
imbalanced_metrics["dataset_version"] = "more_imbalanced"

comparison = pd.concat([original_metrics, imbalanced_metrics], axis=0).reset_index(drop=True)
display(comparison.sort_values(["model", "dataset_version"]))
```

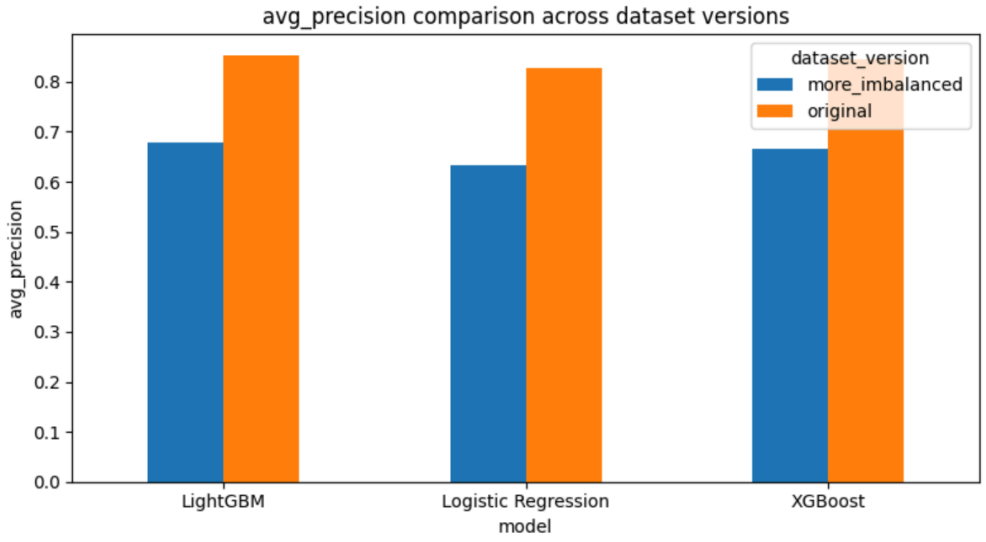
| | model | roc_auc | avg_precision | dataset_version |
|---|---|---|---|---|
| 3 | LightGBM | 0.906765 | 0.678692 | more_imbalanced |
| 0 | LightGBM | 0.910634 | 0.852094 | original |
| 5 | Logistic Regression | 0.898170 | 0.632826 | more_imbalanced |
| 2 | Logistic Regression | 0.899210 | 0.828398 | original |
| 4 | XGBoost | 0.906017 | 0.665281 | more_imbalanced |
| 1 | XGBoost | 0.907914 | 0.846129 | original |

Emmanuel 1011048
GunHyun 1010966
Ian Thomas Matthew 1010766
Nicole 1011069
Submission Date: 25th April, 2026
42.515 - Machine Learning and Analytics
MTD DS Term 2/ 2026

---

```
for metric_name in ["roc_auc", "avg_precision"]:
    pivot_df = comparison.pivot(index="model", columns="dataset_version", values=metric_name)
    pivot_df.plot(kind="bar", figsize=(8, 4.5))
    plt.title(f"{metric_name} comparison across dataset versions")
    plt.ylabel(metric_name)
    plt.xticks(rotation=0)
    plt.tight_layout()
    plt.show()
```



Emmanuel 1011048
GunHyun 1010966
Ian Thomas Matthew 1010766
Nicole 1011069
Submission Date: 25th April, 2026
42.515 - Machine Learning and Analytics
MTD DS Term 2/ 2026

---

## 12. Optional: train on all labeled data and score the provided `test.csv`

This step is **not for evaluation**, because `test.csv` does not include ground-truth labels here.
It is only useful if you want to generate probability predictions for submission or inspection.

```
best_model_name = metrics_df.sort_values("avg_precision", ascending=False).iloc[0]["model"]
best_model_obj = available_models[best_model_name]

final_pipeline = build_pipeline(best_model_obj)
final_pipeline.fit(X, y)

test_features = test_df.copy()
if id_col in test_features.columns:
    test_ids = test_features[id_col].copy()
    test_features = test_features.drop(columns=[id_col])
else:
    test_ids = pd.Series(np.arange(len(test_features)), name="row_id")

test_scores = final_pipeline.predict_proba(test_features)[:, 1]

submission_like_df = pd.DataFrame({
    "CustomerID": test_ids,
    "churn_probability": test_scores
})

display(submission_like_df.head())
```

| | CustomerID | churn_probability |
|---|---|---|
| 0 | 0465e1d5adc84233 | 0.160852 |
| 1 | 2e92a20d5ba44245 | 0.096913 |
| 2 | 3415bc0eabec4c83 | 0.106615 |
| 3 | a9701499e89847d3 | 0.068170 |
| 4 | 808c6d4f17f542d5 | 0.032522 |

## 13. Final interpretation

### What this notebook shows

1. The provided churn dataset already has class imbalance.
2. Models can achieve good **ROC-AUC** even when the class distribution is skewed.
3. **Average Precision / AU-PRC-style evaluation** is often more sensitive to minority-class performance.
4. When the same dataset is made more imbalanced, PR-based evaluation becomes even more informative.

### Assignment takeaway

For imbalanced classification problems, especially when the positive class is the one you care about most, **AU-PRC is often a better reflection of practical model usefulness than ROC-AUC**.